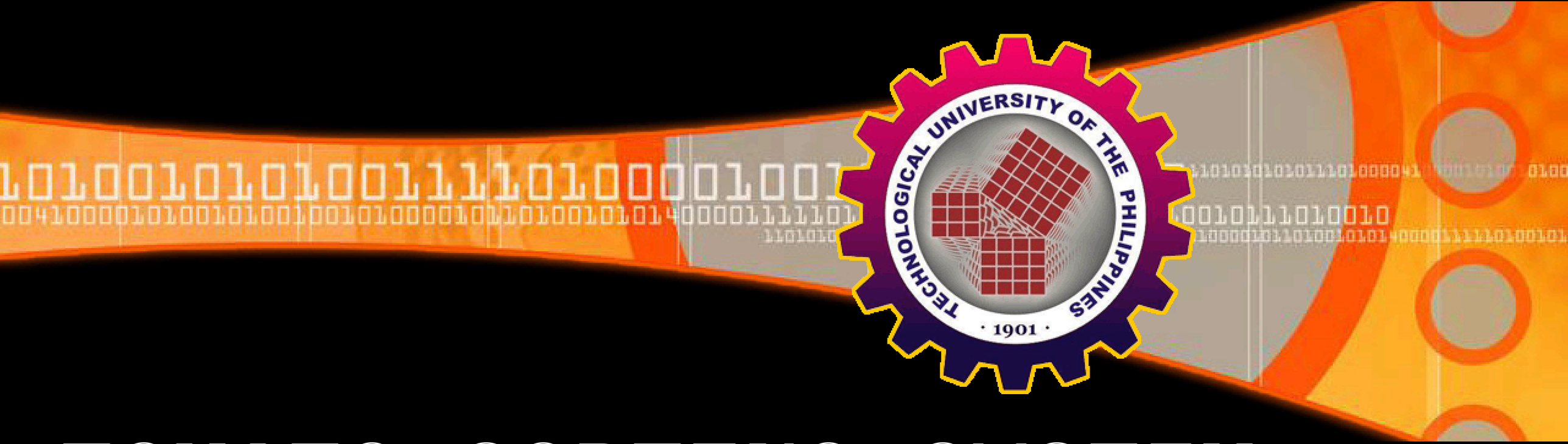
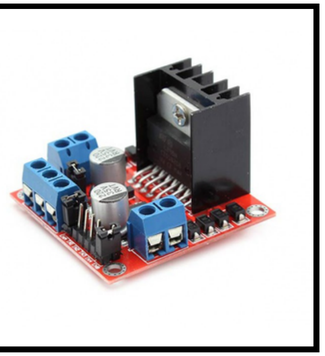


TOMATO SORTING SYSTEM

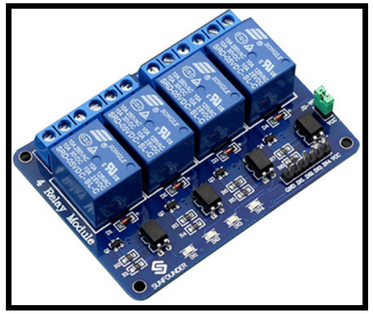
AUTOMATED RIPENESS SORTING SYSTEM

Prepared By:
Bagon
Mandawe
Priol

Materials



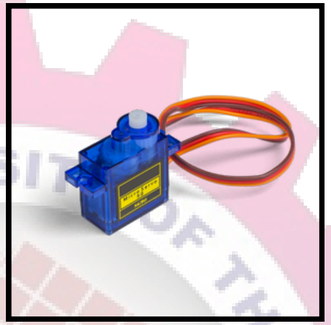
L289N MOTOR DRIVER



5V 4-CHANNEL RELAY MODULE



5V MINI FAN



SG90 MOTOR



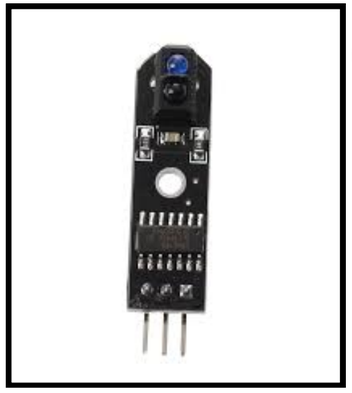
12V DC MOTOR



MG995 MOTOR



MQ-135



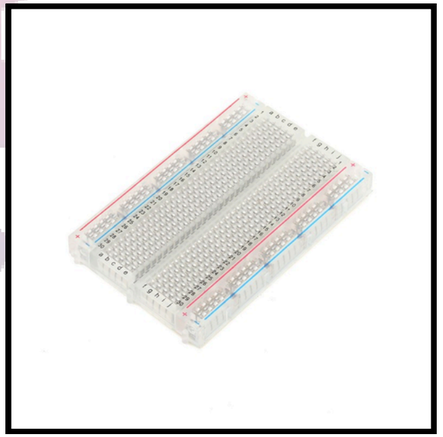
TRCRT5000



ESP32-CAM



ESP32



BREADBOARD

DESCRIPTION

This project will utilize an ESP32 and ESP32-Camera to develop a Tomato Sorting System that aims to check the tomato's ripeness and detect any defect (dark spots or molds) on the tomato and sort them accordingly to different bins.

1. **Defect Analysis**
   - An ESP32-Camera will take and process 4 images for each side of the tomato.
   - An MG995 Servo Motor will rotate the floor on the Camera Chamber for the ESP32-Cam take the 4 images.
   - If no defect is found, then the tomato will be checked by the other sensors and delivered to its respective bin, else the tomato will go to the reject bin.

DESCRIPTION

2. Ripeness Detection

- An MQ-135 Gas Sensor will receive and detect the ethylene gas levels of a tomato to determine its ripeness level (this is also supported by the ESP32-Cam by analyzing the tomato's color.
- A 5V Mini Fan will help in the air flow for the MQ-135 so that it receives the released gas of the tomato.

3. Height Detection

- A VL53L0X Range Sensor will check if there is a sudden huge change in its range readings. If there is, then it will decide the tomato's size (If Sensor2 detects it, then it is large. If Sensor2 did not detect it, then it is small.

MAIN COMPONENTS

Microcontroller

- ESP32 – Handles sensor readings, actuator control, and Wi-Fi connectivity for remote monitoring.

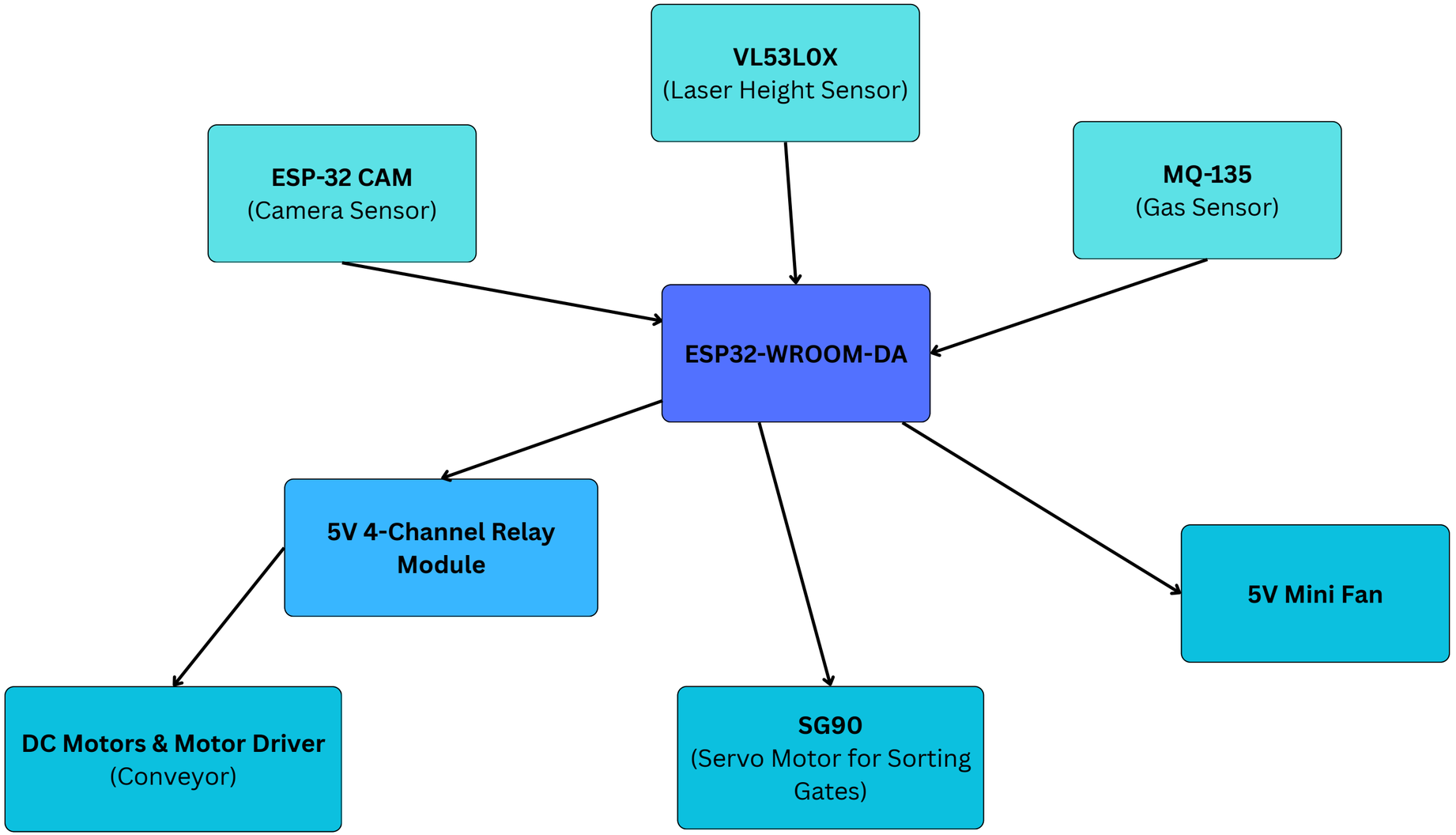
Inputs (Sensors)

- ESP32-Camera – Defect & Color Sensor
- MQ-135 – Gas Sensor
- VL53L0X - Height Sensor

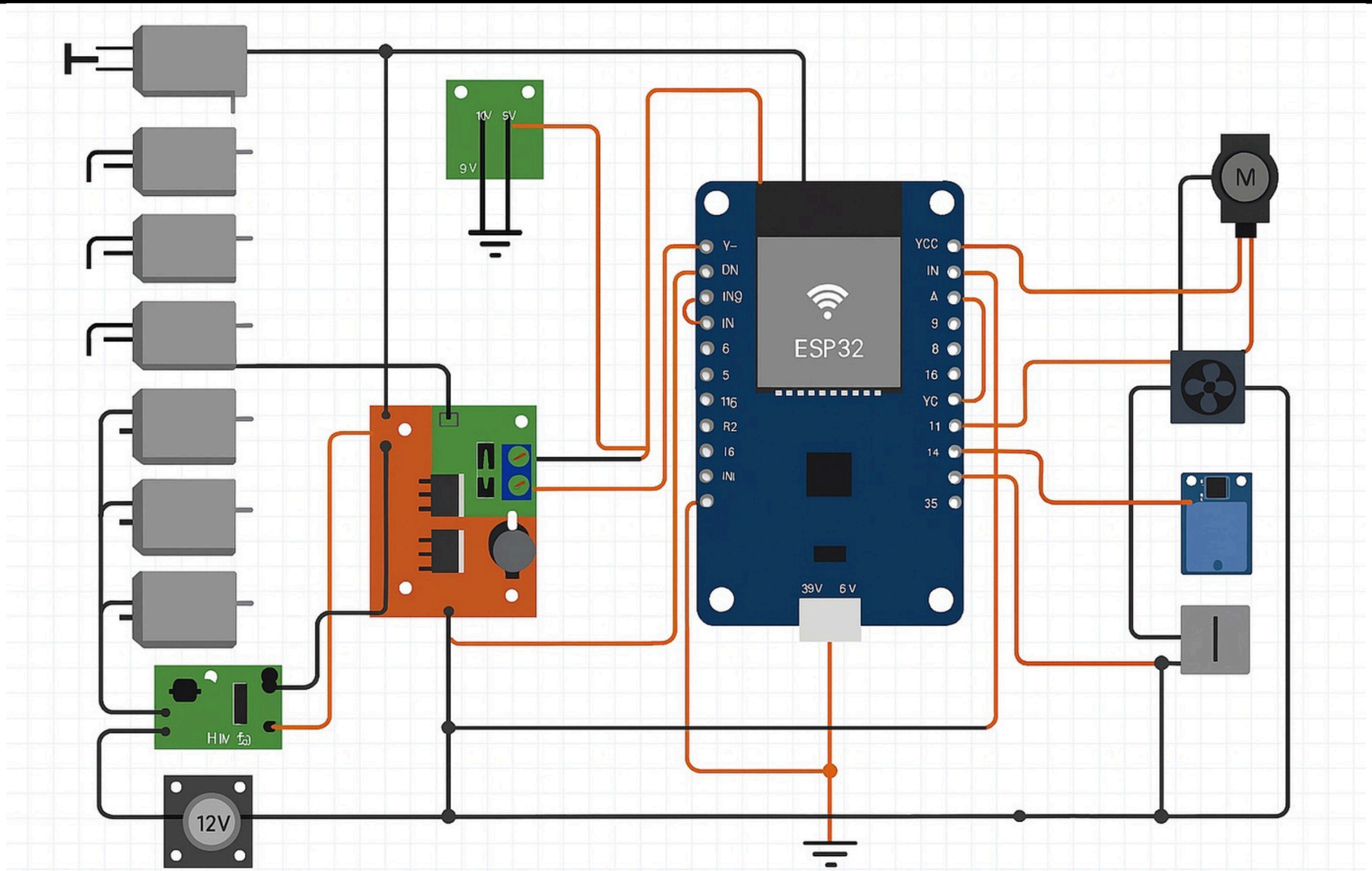
Outputs

- 12V DC Motor – For the Conveyor Belt
- SG90 Motor – For Sorting Gates
- 5V Mini Fan – For Gas Sensor Air Flow Support
- MG995 Motor – For Floor rotating to Help the Camera Sensor

BLOCK DIAGRAM

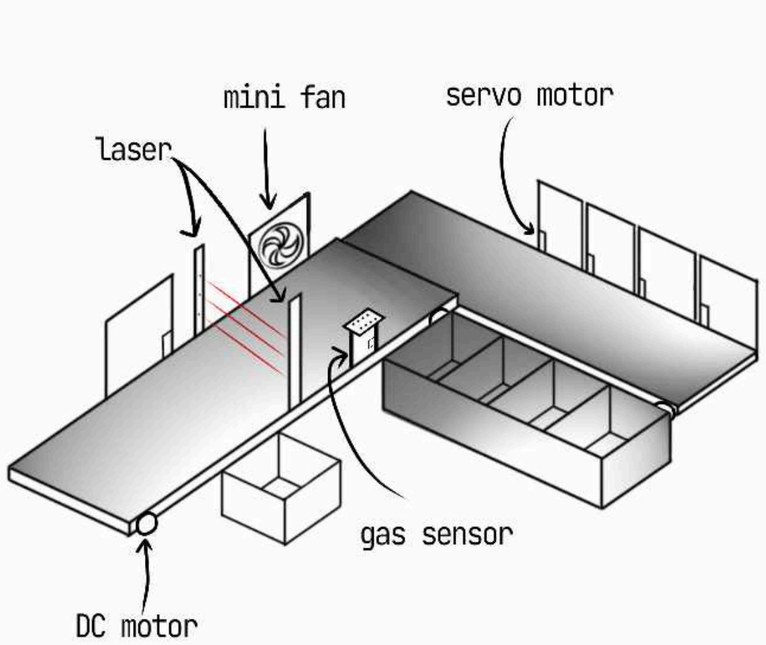


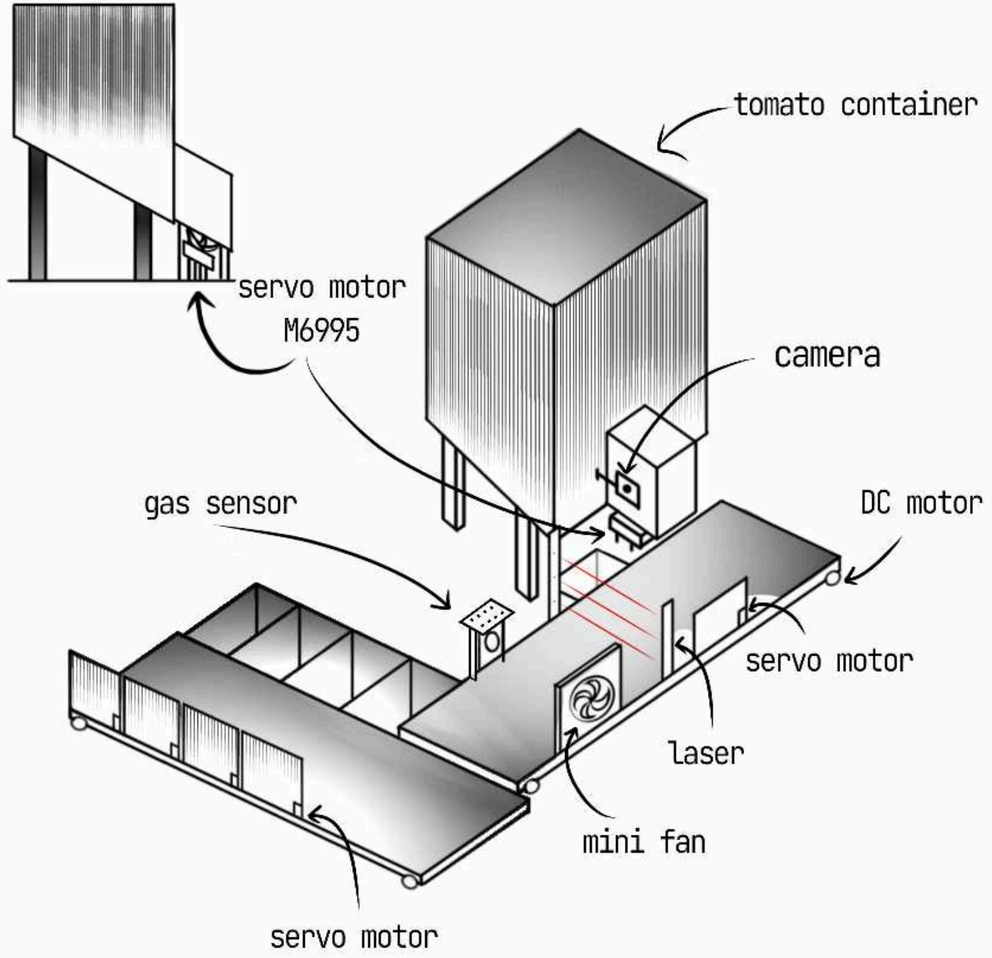
SCHEMATIC DIAGRAM

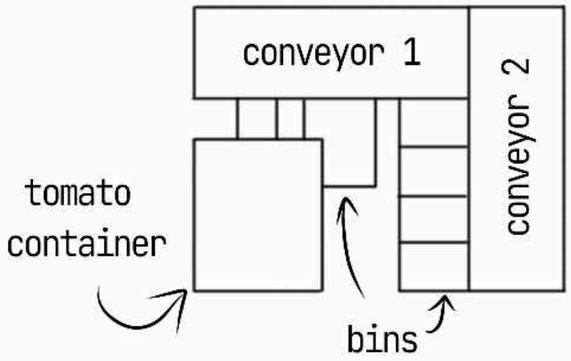


EXPECTED OUPUT

DIGITALIZED DRAWING







CODES

MAIN-ESP32.INO

TEST2ESPCAM.ino

```ino
#include "esp_camera.h"
#include <WiFi.h>
#include <WiFiClient.h>
#include <WebServer.h>
#include <ESPmDNS.h>
#include <HTTPClient.h>

#define CAMERA_MODEL_AI_THINKER // Make sure this matches your camera module

#if defined(CAMERA_MODEL_AI_THINKER)
#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27
#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22
#else
#error "Camera model not selected"
#endif

// Flash LED
#define FLASH_LED_PIN 4

const char* ssid = "HUAWEI-2.4G-k8Zn";
const char* password = "B73TQah5";
const char* mainESP32Address = "http://192.168.100.56/results"; // IP of the main ESP32

WebServer server(80);

// Function to analyze tomato for dark spots
bool analyzeDarkSpots(camera_fb_t *fb) {
  if (!fb) return false;
```

TEST2MAINESP.ino

```ino
#define CONVEYOR1_ENA 32            // Conveyor Belt 1 - Speed Control (PWM)
#define CONVEYOR2_IN3 27            // Conveyor Belt 2 - Direction Control
#define CONVEYOR2_IN4 14            // Conveyor Belt 2 - Direction Control
#define CONVEYOR2_ENB 33            // Conveyor Belt 2 - Speed Control (PWM)
// Sensor pins
#define MQ135_GAS_SENSOR_PIN 34     // MQ-135 Gas Sensor - Analog Gas Detection
#define TCRT5000_HEIGHT_SENSOR_PIN 35 // TCRT5000 Height Sensor - Digital Height Detection
// Other components
#define FAN_GAS_CHAMBER_PIN 18      // Mini Fan (Gas Chamber) - ON/OFF Control
#define RELAY_CONVEYOR1_PIN 19      // Relay IN1 (Conveyor 1 Power Switch)

// Servo objects
Servo stopperGate1;     // Camera chamber stopper
Servo stopperGate2;     // Gas analysis stopper
Servo rotatingFloor;    // MG995 for rotating tomato
Servo gate1;            // Small Ripe sorting gate
Servo gate2;            // Large Ripe sorting gate
Servo gate3;            // Small Unripe sorting gate
Servo gate4;            // Large Unripe sorting gate

// Servo positions
// Stopper Gates
const int STOPPER_GATE1_CLOSED = 90;
const int STOPPER_GATE1_OPEN = 180;
const int STOPPER_GATE2_CLOSED = 85;
const int STOPPER_GATE2_OPEN = 180;

// Sorting Gates - Open positions
const int GATE1_OPEN = 68;
const int GATE2_OPEN = 58;
const int GATE3_OPEN = 65;
const int GATE4_OPEN = 55;

// HTML for the main page with capture button (keep your existing HTML)
const char indexHTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Tomato Analysis System</title>
  <style>
```

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
84      body {
85        font-family: Arial, sans-serif;
86        text-align: center;
87        margin: 0;
88        padding: 20px;
89        background-color: #f5f5f5;
90      }
91      .container {
92        max-width: 800px;
93        margin: 0 auto;
94        background-color: white;
95        padding: 20px;
96        border-radius: 10px;
97        box-shadow: 0 4px 8px rgba(0,0,0,0.1);
98      }
99      h1 {
100       color: #e63946;
101     }
102     .button {
103       background-color: #e63946;
104       color: white;
105       border: none;
106       padding: 15px 32px;
107       text-align: center;
108       text-decoration: none;
109       display: inline-block;
110       font-size: 16px;
111       margin: 20px 0;
112       cursor: pointer;
113       border-radius: 5px;
114       transition: background-color 0.3s;
115     }
116     .button:hover {
117       background-color: #c1121f;
118     }
119     #status {
120       margin: 20px 0;
121       padding: 15px;
122       background-color: #f8f9fa;
123       border-radius: 5px;
124       border-left: 5px solid #6c757d;
```

TEST2MAINESP.ino

```
125     }
126     #results {
127       text-align: left;
128       white-space: pre-wrap;
129       background-color: #f8f9fa;
130       padding: 15px;
131       border-radius: 5px;
132       margin-top: 20px;
133     }
134     .loading {
135       display: none;
136       margin: 20px auto;
137       border: 6px solid #f3f3f3;
138       border-top: 6px solid #e63946;
139       border-radius: 50%;
140       width: 40px;
141       height: 40px;
142       animation: spin 2s linear infinite;
143     }
144     @keyframes spin {
145       0% { transform: rotate(0deg); }
146       100% { transform: rotate(360deg); }
147     }
148   </style>
149 </head>
150 <body>
151   <div class="container">
152     <h1>Tomato Quality Analysis System</h1>
153     <p>Click the button below to capture images and analyze tomato quality</p>
154
155     <button class="button" onclick="captureImage()">Capture Image</button>
156
157     <div id="loading" class="loading"></div>
158
159     <div id="status">Ready to capture</div>
160
161     <div id="results">No analysis results yet</div>
162   </div>
163
164   <script>
165     function captureImage() {
```

CODES

MAIN-ESP32.INO

TEST2ESPCAM.ino

```
165
166   // Combine results
167   bool rejected = hasDarkSpots1 && hasDarkSpots2; // Reject only if both images show dark spots
168   String ripenessResult = (ripeness1 == ripeness2) ? ripeness1 : "Mixed results: " + ripeness1 + " and "
169
170   // Create result string
171   String result = "Tomato Analysis Results:\n";
172   result += "Dark Spots: " + String(rejected ? "Too many (REJECTED)" : "Few or none (ACCEPTED)") + "\n";
173   result += "Ripeness: " + ripenessResult;
174
175   // Log results
176   Serial.println(result);
177
178   // Send results to main ESP32
179   HTTPClient http;
180   http.begin(mainESP32Address);
181   http.addHeader("Content-Type", "text/plain");
182   int httpResponseCode = http.POST(result);
183
184   if (httpResponseCode > 0) {
185     Serial.print("HTTP Response code: ");
186     Serial.println(httpResponseCode);
187   }
188   else {
189     Serial.print("Error code: ");
190     Serial.println(httpResponseCode);
191   }
192   http.end();
193
194   // Free the memory
195   esp_camera_fb_return(fb1);
196   esp_camera_fb_return(fb2);
197 }
198
199 void handleCapture() {
200   server.send(200, "text/plain", "Capturing images and analyzing tomato...");
201   captureAndAnalyze();
202 }
203
204 void setup() {
205   Serial.begin(115200);
206
207   // Configure flash LED pin
```

TEST2ESPCAM.ino

```
208   pinMode(FLASH_LED_PIN, OUTPUT);
209   digitalWrite(FLASH_LED_PIN, LOW);
210
211   // Camera configuration
212   camera_config_t config;
213   config.ledc_channel = LEDC_CHANNEL_0;
214   config.ledc_timer = LEDC_TIMER_0;
215   config.pin_d0 = Y2_GPIO_NUM;
216   config.pin_d1 = Y3_GPIO_NUM;
217   config.pin_d2 = Y4_GPIO_NUM;
218   config.pin_d3 = Y5_GPIO_NUM;
219   config.pin_d4 = Y6_GPIO_NUM;
220   config.pin_d5 = Y7_GPIO_NUM;
221   config.pin_d6 = Y8_GPIO_NUM;
222   config.pin_d7 = Y9_GPIO_NUM;
223   config.pin_xclk = XCLK_GPIO_NUM;
224   config.pin_pclk = PCLK_GPIO_NUM;
225   config.pin_vsync = VSYNC_GPIO_NUM;
226   config.pin_href = HREF_GPIO_NUM;
227   config.pin_sscb_sda = SIOD_GPIO_NUM;
228   config.pin_sscb_scl = SIOC_GPIO_NUM;
229   config.pin_pwdn = PWDN_GPIO_NUM;
230   config.pin_reset = RESET_GPIO_NUM;
231   config.xclk_freq_hz = 20000000;
232   config.pixel_format = PIXFORMAT_JPEG;
233   config.frame_size = FRAMESIZE_VGA; // Use VGA for better analysis
234   config.jpeg_quality = 10; // Higher quality for better analysis
235   config.fb_count = 2; // Use 2 framebuffers for better performance
236
237   // Initialize camera
238   esp_err_t err = esp_camera_init(&config);
239   if (err != ESP_OK) {
240     Serial.printf("Camera init failed with error 0x%x", err);
241     return;
242   }
243
244   // Set higher contrast and saturation for better color detection
245   sensor_t * s = esp_camera_sensor_get();
246   s->set_contrast(s, 2);      // Increase contrast
247   s->set_saturation(s, 2);    // Increase saturation for better color detection
248   s->set_brightness(s, 1);    // Slightly increase brightness
249
```

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
250  }
251
252  // Function to direct tomato to Bin 3 (Small Unripe)
253  void directToBin3() {
254    Serial.println("Directing tomato to Bin 3 (Small Unripe)");
255    // Open all gates first
256    openAllSortingGates();
257    delay(1000);
258
259    // Close Gates 1, 2 (for support) and 3
260    gate1.write(65); // Support position
261    delay(1500);
262    gate2.write(43); // Support position
263    delay(1500);
264    gate3.write(25);
265    Serial.println("Gates positioned - Tomato directed to Small Unripe bin");
266
267    // Wait 30 seconds before resetting (NEW)
268    delay(30000);
269  }
270
271  // Function to direct tomato to Bin 4 (Large Unripe)
272  void directToBin4() {
273    Serial.println("Directing tomato to Bin 4 (Large Unripe)");
274    // Open all gates first
275    openAllSortingGates();
276    delay(1000);
277
278    // Close Gates 1, 2, 3 (for support) and 4
279    gate1.write(50); // Support position
280    delay(1500);
281    gate2.write(55); // Support position
282    delay(1500);
283    gate3.write(55); // Support position
284    delay(1500);
285    gate4.write(3);
286    Serial.println("Gates positioned - Tomato directed to Large Unripe bin");
287
288    // Wait 30 seconds before resetting (NEW)
289    delay(30000);
290  }
291
292  // Function for rejected tomatoes - they go straight to reject bin
293  void rejectTomato() {
```

TEST2MAINESP.ino

```
294    Serial.println("Tomato REJECTED - Directing to reject bin");
295    // Open all gates so tomato passes through all sorting bins
296    openAllSortingGates();
297
298    // Wait 30 seconds before any other actions (NEW)
299    delay(30000);
300  }
301
302  // Function to check tomato size using TCRT5000 sensor
303  bool checkTomatoSize() {
304    // Read the TCRT5000 height sensor
305    // If sensor detects object (returns LOW), tomato is large
306    int sensorValue = digitalRead(TCRT5000_HEIGHT_SENSOR_PIN);
307    bool isLarge = (sensorValue == LOW);
308
309    Serial.print("Tomato size check: ");
310    Serial.println(isLarge ? "LARGE" : "SMALL");
311
312    return isLarge;
313  }
314
315  // Function to check tomato ripeness using MQ135 gas sensor
316  bool checkTomatoRipeness() {
317    // Read the MQ135 gas sensor value
318    int gasValue = analogRead(MQ135_GAS_SENSOR_PIN);
319
320    // Define threshold for ethylene gas (adjust based on calibration)
321    const int ETHYLENE_THRESHOLD = 2000; // Example threshold, adjust as needed
322
323    bool isRipe = (gasValue > ETHYLENE_THRESHOLD);
324
325    Serial.print("Gas sensor reading: ");
326    Serial.print(gasValue);
327    Serial.print(" - Tomato ripeness: ");
328    Serial.println(isRipe ? "RIPE" : "UNRIPE");
329
330    return isRipe;
331  }
332
333  // Start tomato processing in camera chamber
334  void startTomatoProcessing() {
335    Serial.println("\n----- STARTING TOMATO PROCESSING -----");
336
```

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
338   // Close stopper gate 1 to be ready for next tomato
339   stopperGate1.write(STOPPER_GATE1_CLOSED);
340
341   // Check ripeness with gas sensor
342   // Turn on fan to circulate air around tomato
343   digitalWrite(FAN_GAS_CHAMBER_PIN, HIGH);
344
345   // Wait for gas analysis (15 seconds with gate 2 closed)
346   Serial.println("Performing gas analysis...");
347   delay(15000);
348
349   // Get ripeness data from gas sensor
350   bool gasRipenessResult = checkTomatoRipeness();
351
352   // Turn off fan
353   digitalWrite(FAN_GAS_CHAMBER_PIN, LOW);
354
355   // Open gate 2 to let tomato proceed
356   Serial.println("Opening gas chamber gate");
357   stopperGate2.write(STOPPER_GATE2_OPEN);
358
359   // Start conveyor belt 1 again to move tomato to conveyor belt 2 (increased from 5s to 10s)
360   digitalWrite(RELAY_CONVEYOR1_PIN, HIGH);
361   delay(10000);
362   digitalWrite(RELAY_CONVEYOR1_PIN, LOW);
363
364   // Reset gate 2 for next tomato
365   stopperGate2.write(STOPPER_GATE2_CLOSED);
366
367   // Decide sorting bin based on size, ripeness and acceptance
368   if (!tomatoAccepted) {
369     // If tomato is rejected, direct to reject bin
370     rejectTomato();
371   } else {
372     // Direct to appropriate sorting bin based on size and ripeness
373     if (tomatoIsRipe) {
374       if (tomatoIsLarge) {
375         directToBin2(); // Large Ripe
376       } else {
377         directToBin1(); // Small Ripe
378       }
379     } else {
380       if (tomatoIsLarge) {
```

TEST2MAINESP.ino

```
381         directToBin4(); // Large Unripe
382       } else {
383         directToBin3(); // Small Unripe
384       }
385     }
386   }
387
388   // Wait for tomato to reach bin (increased from 8s to 15s)
389   delay(15000);
390
391   // Reset all gates to default positions
392   openAllSortingGates();
393   stopperGate1.write(STOPPER_GATE1_CLOSED);
394   stopperGate2.write(STOPPER_GATE2_CLOSED);
395
396   Serial.println("----- TOMATO PROCESSING COMPLETE -----");
397 }
398
399 // Handle root URL request
400 void handleRoot() {
401   server.send(200, "text/html", indexHTML);
402 }
403
404 // Handle capture trigger from web interface
405 void handleTriggerCapture() {
406
407   if(WiFi.status() != WL_CONNECTED) {
408     server.send(503, "text/plain", "Camera offline");
409     return;
410   }
411
412   // Reset the results
413   analysisResults = "Waiting for results...";
414
415   Serial.println("Capture button pressed - Starting capture sequence");
416
417   // Send a request to the ESP32-CAM to capture and analyze images
418   HTTPClient http;
419   String url = "http://" + String(esp32CamIP) + "/capture";
420
421   http.begin(url);
422   int httpCode = http.GET();
423
```

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
424     if (httpCode > 0) {
425       String response = http.getString();
426       server.send(200, "text/plain", "Request sent to camera. Waiting for analysis...");
427     } else {
428       server.send(500, "text/plain", "Failed to connect to the camera. Please check if it's online.");
429     }
430
431     http.end();
432   }
433
434   // Handle results retrieval from web interface
435   void handleGetResults() {
436     server.send(200, "text/plain", analysisResults);
437   }
438
439   // Handle incoming results from ESP32-CAM
440   // void handleResults() {
441   //   // This endpoint receives results from the ESP32-CAM
442   //   if (server.hasArg("plain")) {
443   //     analysisResults = server.arg("plain");
444   //     int currentGas = analogRead(MQ135_GAS_SENSOR_PIN);
445   //
446   //      tomatoLog[tomatoCount].gasValue = currentGas;
447   //
448   //     // Print the analysis results to serial monitor
449   //     Serial.println("\n----- TOMATO ANALYSIS RESULTS -----");
450   //     Serial.println(analysisResults);
451   //     Serial.println("-----------------------------------");
452   //
453   //     // Parse results to determine if tomato is accepted and its ripeness
454   //     tomatoAccepted = analysisResults.indexOf("REJECTED") == -1;
455   //     tomatoIsRipe = analysisResults.indexOf("Ripe (Red-Orange)") >= 0;
456   //
457   //     // Start the tomato processing sequence
458   //     startTomatoProcessing();
459   //
460   //     server.send(200, "text/plain", "Results received");
461   //   } else {
462   //     server.send(400, "text/plain", "No data received");
463   //   }
464   // }
465
466   void printWiFiStatus(int status) {
```

TEST2MAINESP.ino

```
467     switch (status) {
468       case WL_CONNECTED:
469         Serial.println("Connected");
470         break;
471       case WL_NO_SHIELD:
472         Serial.println("No WiFi shield/module present");
473         break;
474       case WL_IDLE_STATUS:
475         Serial.println("Idle");
476         break;
477       case WL_NO_SSID_AVAIL:
478         Serial.println("No SSID available - network not found");
479         break;
480       case WL_SCAN_COMPLETED:
481         Serial.println("Scan completed");
482         break;
483       case WL_CONNECT_FAILED:
484         Serial.println("Connection failed - wrong password?");
485         break;
486       case WL_CONNECTION_LOST:
487         Serial.println("Connection lost");
488         break;
489       case WL_DISCONNECTED:
490         Serial.println("Disconnected");
491         break;
492       default:
493         Serial.print("Unknown: ");
494         Serial.println(status);
495         break;
496     }
497   }
498
499   void setupConveyorBelts() {
500     // Set up conveyor belt pins
501     pinMode(CONVEYOR1_IN1, OUTPUT);
502     pinMode(CONVEYOR1_IN2, OUTPUT);
503     pinMode(CONVEYOR1_ENA, OUTPUT);
504     pinMode(CONVEYOR2_IN3, OUTPUT);
505     pinMode(CONVEYOR2_IN4, OUTPUT);
506     pinMode(CONVEYOR2_ENB, OUTPUT);
507
508     // Set initial directions
509     digitalWrite(CONVEYOR1_IN1, HIGH);
```

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
510     digitalWrite(CONVEYOR2_IN4, LOW);
511
512     // Set initial speeds
513     analogWrite(CONVEYOR1_ENA, 200); // Speed value 0-255
514     analogWrite(CONVEYOR2_ENB, 200); // Speed value 0-255
515
516     // Set up relay to control conveyor 1
517     pinMode(RELAY_CONVEYOR1_PIN, OUTPUT);
518       digitalWrite(RELAY_CONVEYOR1_PIN, HIGH); // Conveyor belt 1 is always ON
519
520
521     Serial.println("Conveyor belts initialized");
522   }
523
524   void setupSensorsAndActuators() {
525     // Set up sensors
526     pinMode(MQ135_GAS_SENSOR_PIN, INPUT);
527     pinMode(TCRT5000_HEIGHT_SENSOR_PIN, INPUT);
528
529     // Set up fan
530     pinMode(FAN_GAS_CHAMBER_PIN, OUTPUT);
531     digitalWrite(FAN_GAS_CHAMBER_PIN, LOW); // Start with fan off
532
533     Serial.println("Sensors and actuators initialized");
534   }
535
536   void setupServos() {
537     // Attach all servos
538     ESP32PWM::allocateTimer(0);
539     ESP32PWM::allocateTimer(1);
540     ESP32PWM::allocateTimer(2);
541     ESP32PWM::allocateTimer(3);
542
543     // Set servo PWM frequency
544     stopperGate1.setPeriodHertz(50);
545     stopperGate2.setPeriodHertz(50);
546     rotatingFloor.setPeriodHertz(50);
547     gate1.setPeriodHertz(50);
548     gate2.setPeriodHertz(50);
549     gate3.setPeriodHertz(50);
550     gate4.setPeriodHertz(50);
```

TEST2MAINESP.ino

```
552     // Attach servos to pins
553     stopperGate1.attach(SG90_STOPPER_GATE1_PIN, 500, 2400);
554     stopperGate2.attach(SG90_STOPPER_GATE2_PIN, 500, 2400);
555     rotatingFloor.attach(MG995_ROTATING_FLOOR_PIN, 500, 2400);
556     gate1.attach(SG90_GATE1_PIN, 500, 2400);
557     gate2.attach(SG90_GATE2_PIN, 500, 2400);
558     gate3.attach(SG90_GATE3_PIN, 500, 2400);
559     gate4.attach(SG90_GATE4_PIN, 500, 2400);
560
561     // Move to initial positions
562     stopperGate1.write(STOPPER_GATE1_CLOSED);
563     stopperGate2.write(STOPPER_GATE2_CLOSED);
564     rotatingFloor.write(0);
565
566     // Set all sorting gates to open position
567     openAllSortingGates();
568
569     Serial.println("Servos initialized");
570   }
571
572   int countRipe();
573   int countUnripe();
574   int countMixed();
575   int countAccepted();
576   int countRejected();
577   String extractRipeness(String data);
578   String getTimeStamp() {
579     // Get current time
580     unsigned long currentMillis = millis();
581     unsigned long seconds = currentMillis / 1000;
582     unsigned long minutes = seconds / 60;
583     unsigned long hours = minutes / 60;
584
585     // Format as HH:MM:SS
586     String timeStamp = "";
587
588     // Hours (padded with leading zero if needed)
589     if (hours % 24 < 10) {
590       timeStamp += "0";
591     }
592     timeStamp += String(hours % 24) + ":";
```

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
562     stopperGate1.write(STOPPER_GATE1_CLOSED);
563     stopperGate2.write(STOPPER_GATE2_CLOSED);
564     rotatingFloor.write(0);
565
566     // Set all sorting gates to open position
567     openAllSortingGates();
568
569     Serial.println("Servos initialized");
570   }
571
572   int countRipe();
573   int countUnripe();
574   int countMixed();
575   int countAccepted();
576   int countRejected();
577   String extractRipeness(String data);
578   String getTimeStamp() {
579     // Get current time
580     unsigned long currentMillis = millis();
581     unsigned long seconds = currentMillis / 1000;
582     unsigned long minutes = seconds / 60;
583     unsigned long hours = minutes / 60;
584
585     // Format as HH:MM:SS
586     String timeStamp = "";
587
588     // Hours (padded with leading zero if needed)
589     if (hours % 24 < 10) {
590       timeStamp += "0";
591     }
592     timeStamp += String(hours % 24) + ":";
593
594     // Minutes (padded with leading zero if needed)
595     if ((minutes % 60) < 10) {
596       timeStamp += "0";
597     }
598     timeStamp += String(minutes % 60) + ":";
599
600     // Seconds (padded with leading zero if needed)
601     if ((seconds % 60) < 10) {
602       timeStamp += "0";
```

TEST2MAINESP.ino

```
606     return timeStamp;
607   }
608   void handleResults();  // Add this line
609
610   void setup() {
611     Serial.begin(115200);
612     delay(1000);  // Give serial time to initialize
613
614     Serial.println("\n\n----- ESP32 Tomato Analysis System -----");
615     if(!SPIFFS.begin(true)){
616     Serial.println("SPIFFS Mount Failed");
617   }
618
619     // Add to setup():
620   server.on("/manual-sort", HTTP_GET, [](){
621     String type = server.arg("type");
622     // Implement manual sorting logic
623   });
624
625     // Initialize hardware
626     setupServos();
627     setupConveyorBelts();
628     setupSensorsAndActuators();
629
630     // Set up WiFi
631     WiFi.disconnect(true);  // Disconnect and clear stored settings
632     WiFi.persistent(false); // Don't save settings to flash
633     delay(1000);
634
635     Serial.print("ESP32 MAC Address: ");
636     Serial.println(WiFi.macAddress());
637
638     // Set WiFi mode explicitly
639     WiFi.mode(WIFI_STA);
640     delay(100);
641
642     // Scan for networks
643     int networks = WiFi.scanNetworks();
644     if (networks > 0) {
645       Serial.print(networks);
646       Serial.println(" networks found:");
647       for (int i = 0; i < networks; i++
```

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
    Serial.print("  ");
    Serial.print(i + 1);
    Serial.print(": ");
    Serial.print(WiFi.SSID(i));
    Serial.print(" (");
    Serial.print(WiFi.RSSI(i));
    Serial.println(" dBm)");
  }
}

// Connect to WiFi
Serial.println("Connecting to WiFi...");
WiFi.begin(ssid, password);

unsigned long startAttemptTime = millis();
while (WiFi.status() != WL_CONNECTED && millis() - startAttemptTime < 20000) {
  Serial.print(".");
  delay(500);
}
Serial.println("");

if (WiFi.status() == WL_CONNECTED) {
  Serial.println("WiFi connected successfully!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  // Configure web server routes
  server.on("/", HTTP_GET, handleRoot);
  server.on("/trigger-capture", HTTP_GET, handleTriggerCapture);
  server.on("/get-results", HTTP_GET, handleGetResults);
  server.on("/results", HTTP_POST, handleResults);

  // Start the server
  server.begin();
  server.on("/data", HTTP_GET, handleDataPage);
  Serial.println("HTTP server started");
  Serial.println("System ready - waiting for tomato analysis");
} else {
  Serial.println("WiFi connection failed!");
  printWiFiStatus(WiFi.status());
}
```

TEST2MAINESP.ino

```
    .timeout_ms = 2000,  // 5 second timeout
    .idle_core_mask = 0, // Check both cores
    .trigger_panic = true // Trigger panic on timeout
  };
  esp_task_wdt_init(&twdt_config);
  esp_task_wdt_add(NULL);
}

void handleTomatoProcessing() {
  if (!processingActive) return;

  unsigned long currentMillis = millis();
  unsigned long stateElapsedTime = currentMillis - currentStateStartTime;

  switch (processingState) {
    case 1: // Check size and prepare for rotation
      if (stateElapsedTime >= 100) { // Short delay for size check
        tomatoIsLarge = checkTomatoSize();
        Serial.println("Rotating tomato for second image capture");
        rotatingFloor.write(180);
        processingState = 2;
        currentStateStartTime = currentMillis;
      }
      break;

    case 2: // Wait for second image
      if (stateElapsedTime >= 2000) {
        rotatingFloor.write(0);
        Serial.println("Waiting for image processing to complete...");
        processingState = 3;
        currentStateStartTime = currentMillis;
      }
      break;

    case 3: // Wait for image processing
      if (stateElapsedTime >= 30000) {
        Serial.println("Opening camera chamber gate");
        stopperGate1.write(STOPPER_GATE1_OPEN);
        Serial.println("Starting conveyor belt 1");
        digitalWrite(RELAY_CONVEYOR1_PIN, HIGH);
        processingState = 4;
        currentStateStartTime = curre
```

Compiling sketch...

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
735
736   case 4: // Wait for conveyor belt movement
737     if (stateElapsedTime >= 20000) {
738       digitalWrite(RELAY_CONVEYOR1_PIN, LOW);
739       Serial.println("Conveyor belt 1 stopped");
740       stopperGate1.write(STOPPER_GATE1_CLOSED);
741       digitalWrite(FAN_GAS_CHAMBER_PIN, HIGH);
742       Serial.println("Performing gas analysis...");
743       processingState = 5;
744       currentStateStartTime = currentMillis;
745     }
746     break;
747
748   case 5: // Wait for gas analysis
749     if (stateElapsedTime >= 15000) {
750       bool gasRipenessResult = checkTomatoRipeness();
751       digitalWrite(FAN_GAS_CHAMBER_PIN, LOW);
752       Serial.println("Opening gas chamber gate");
753       stopperGate2.write(STOPPER_GATE2_OPEN);
754       digitalWrite(RELAY_CONVEYOR1_PIN, HIGH);
755       processingState = 6;
756       currentStateStartTime = currentMillis;
757     }
758     break;
759
760   case 6: // Wait for tomato to move to conveyor belt 2
761     if (stateElapsedTime >= 10000) {
762       digitalWrite(RELAY_CONVEYOR1_PIN, LOW);
763       stopperGate2.write(STOPPER_GATE2_CLOSED);
764
765       // Decide sorting bin
766       if (!tomatoAccepted) {
767         rejectTomatoStart();
768       } else if (tomatoIsRipe) {
769         if (tomatoIsLarge) {
770           directToBin2Start(); // Large Ripe
771         } else {
772           directToBin1Start(); // Small Ripe
773         }
774       } else {
775         if (tomatoIsLarge) {
776           directToBin4Start(); // L
777         } else {
```

Compiling sketch...

TEST2MAINESP.ino

```
778           directToBin3Start(); // Small Unripe
779         }
780       }
781       processingState = 7;
782       currentStateStartTime = currentMillis;
783     }
784     break;
785
786   case 7: // Wait for tomato to reach bin
787     if (stateElapsedTime >= 30000) {
788       openAllSortingGates();
789       stopperGate1.write(STOPPER_GATE1_CLOSED);
790       stopperGate2.write(STOPPER_GATE2_CLOSED);
791       Serial.println("----- TOMATO PROCESSING COMPLETE -----");
792       processingActive = false; // End processing
793     }
794     break;
795   }
796 }
797
798 void loop() {
799   // Handle web server clients
800   esp_task_wdt_reset();
801   if (WiFi.status() == WL_CONNECTED) {
802     server.handleClient();
803   }
804
805   // Monitor WiFi status
806   static int previousStatus = WiFi.status();
807   int currentStatus = WiFi.status();
808
809   if (currentStatus != previousStatus) {
810     Serial.print("WiFi status changed to: ");
811     printWiFiStatus(currentStatus);
812     previousStatus = currentStatus;
813
814     // If reconnected, restart the web server
815     if (previousStatus != WL_CONNECTED && currentStatus == WL_CONNECTED) {
816       Serial.print("Reconnected to WiFi. IP Address: ");
817       Serial.println(WiFi.localIP());
818       server.begin();
819       Serial.println("HTTP server re
820     }
```

Compiling sketch...

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
  static unsigned long previousMicros = 0;
  unsigned long currentMicros = micros();
  if (currentMicros - previousMicros >= 1000) { // 1ms non-blocking delay
    previousMicros = currentMicros;
  } // Small delay to prevent watchdog timer issues
}

// Fix for String operations on char arrays
// Replace these string operation errors in handleDataPage()
void handleDataPage() {
  String html = R"(
  <!DOCTYPE html>
  <html>
  <head>
    <title>Tomato Analysis Dashboard</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <style>
      body { font-family: Arial; margin: 20px; }
      .grid { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; }
      table { width: 100%; border-collapse: collapse; margin-top: 20px; }
      th, td { padding: 8px; border: 1px solid #ddd; text-align: left; }
      .rejected { background-color: #ffcccc; }
      .accepted { background-color: #ccffcc; }
      .chart-container { background: white; padding: 15px; border-radius: 8px; box-shadow: 0 2px 4px rg
    </style>
  </head>
  <body>
    <h1>Tomato Analysis History</h1>
    <div class="grid">
      <div class="chart-container">
        <h3>Ripeness Distribution</h3>
        <canvas id="ripenessChart"></canvas>
      </div>
      <div class="chart-container">
        <h3>Quality Control</h3>
        <canvas id="defectsChart"></canvas>
      </div>
    </div>
    <h2>Analysis Records</h2>
    <table>
      <tr>
        <th>Time</th>
        <th>Size</th>
```

Uploading...

TEST2MAINESP.ino

```
        <th>Ripeness</th>
        <th>Quality</th>
        <th>Gas Value</th>
      </tr>
  )";

  // Add table rows with tomato data
  for (int i = 0; i < tomatoCount; i++) {
    html += "<tr class='";
    html += tomatoLog[i].rejected ? "rejected" : "accepted";
    html += "'>";
    html += "<td>";
    html += tomatoLog[i].timestamp;
    html += "</td>";
    html += "<td>";
    html += tomatoLog[i].isLarge ? "Large" : "Small";
    html += "</td>";
    html += "<td>";
    html += tomatoLog[i].ripeness;
    html += "</td>";
    html += "<td>";
    html += tomatoLog[i].rejected ? "REJECTED" : "Accepted";
    html += "</td>";
    html += "<td>";
    html += String(tomatoLog[i].gasValue);
    html += "</td>";
    html += "</tr>";
  }

  html += R"(
    </table>
    <script>
      // Ripeness Pie Chart
      new Chart(document.getElementById('ripenessChart'), {
        type: 'pie',
        data: {
          labels: ['Ripe', 'Unripe', 'Mixed'],
          datasets: [{
            data: [)";
  html += String(countRipe()) + "," + String(countUnripe()) + "," + String(countMixed());
  html += R"(],
            backgroundColor: ['#4CAF50', '#F44336', '#FFC107']
          }]
```

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
910           options: { responsive: true }
911         });
912
913         // Defects Bar Chart
914         new Chart(document.getElementById('defectsChart'), {
915           type: 'bar',
916           data: {
917             labels: ['Accepted', 'Rejected'],
918             datasets: [{
919               label: 'Tomatoes',
920               data: [)";
921   html += String(countAccepted()) + "," + String(countRejected());
922   html += R"(],
923               backgroundColor: ['#4CAF50', '#F44336']
924             }]
925           },
926           options: { responsive: true }
927         });
928       </script>
929   </body>
930   </html>
931   )";
932
933   server.send(200, "text/html", html);
934 }
935
936 // Fix for indexOf on char arrays
937 // Replace these functions with strcasestr implementation
938 int countRipe() {
939   int count = 0;
940   for (int i = 0; i < tomatoCount; i++) {
941     if (strstr(tomatoLog[i].ripeness, "Ripe") != NULL) count++;
942   }
943   return count;
944 }
945
946 int countUnripe() {
947   int count = 0;
948   for (int i = 0; i < tomatoCount; i++) {
949     if (strstr(tomatoLog[i].ripeness, "Unripe") != NULL) count++;
950   }
951   return count;
952 }
```

TEST2MAINESP.ino

```
954 int countMixed() {
955   int count = 0;
956   for (int i = 0; i < tomatoCount; i++) {
957     if (strstr(tomatoLog[i].ripeness, "Mixed") != NULL) count++;
958   }
959   return count;
960 }
961
962 int countAccepted() {
963   int count = 0;
964   for (int i = 0; i < tomatoCount; i++) {
965     if (!tomatoLog[i].rejected) count++;
966   }
967   return count;
968 }
969
970 int countRejected() {
971   int count = 0;
972   for (int i = 0; i < tomatoCount; i++) {
973     if (tomatoLog[i].rejected) count++;
974   }
975   return count;
976 }
977
978 // Fix for string extraction function
979 String extractRipeness(String data) {
980   int start = data.indexOf("Ripeness:") + 9;
981   if (start < 9) return "Unknown";
982   int end = data.indexOf(",", start);
983   if (end == -1) end = data.length();
984   return data.substring(start, end);
985 }
986
987 // Fix for handleResults redefinition and struct assignment issues
988 // Remove the second handleResults() function and modify the first one:
989 void handleResults() {
990   // This endpoint receives results from the ESP32-CAM
991   if (server.hasArg("plain")) {
992     analysisResults = server.arg("plain");
993     int currentGas = analogRead(MQ135_GAS_SENSOR_PIN);
994
995     // Store the analysis data
996     if (tomatoCount < MAX_TOMATOES) {
```

CODES

MAIN-ESP32.INO

TEST2MAINESP.ino

```
    tomatoLog[tomatoCount].rejected = (analysisResults.indexOf("REJECTED") >= 0);

    // Copy ripeness string safely to char array
    String ripenessStr = extractRipeness(analysisResults);
    strncpy(tomatoLog[tomatoCount].ripeness, ripenessStr.c_str(), sizeof(tomatoLog[tomatoCount].ripen
    tomatoLog[tomatoCount].ripeness[sizeof(tomatoLog[tomatoCount].ripeness) - 1] = '\0'; // Ensure nu

    tomatoLog[tomatoCount].isLarge = checkTomatoSize();

    // Get timestamp and copy to char array
    String timestampStr = getTimeStamp();
    strncpy(tomatoLog[tomatoCount].timestamp, timestampStr.c_str(), sizeof(tomatoLog[tomatoCount].tim
    tomatoLog[tomatoCount].timestamp[sizeof(tomatoLog[tomatoCount].timestamp) - 1] = '\0'; // Ensure

    tomatoLog[tomatoCount].gasValue = currentGas;
    tomatoCount++;
  } else {
    // Circular buffer if we reach max capacity
    for (int i = 0; i < MAX_TOMATOES-1; i++) {
      tomatoLog[i] = tomatoLog[i+1];
    }

    // Set values for the last record
    tomatoLog[MAX_TOMATOES-1].rejected = (analysisResults.indexOf("REJECTED") >= 0);

    String ripenessStr = extractRipeness(analysisResults);
    strncpy(tomatoLog[MAX_TOMATOES-1].ripeness, ripenessStr.c_str(), sizeof(tomatoLog[MAX_TOMATOES-1]
    tomatoLog[MAX_TOMATOES-1].ripeness[sizeof(tomatoLog[MAX_TOMATOES-1].ripeness) - 1] = '\0';

    tomatoLog[MAX_TOMATOES-1].isLarge = checkTomatoSize();

    String timestampStr = getTimeStamp();
    strncpy(tomatoLog[MAX_TOMATOES-1].timestamp, timestampStr.c_str(), sizeof(tomatoLog[MAX_TOMATOES-
    tomatoLog[MAX_TOMATOES-1].timestamp[sizeof(tomatoLog[MAX_TOMATOES-1].timestamp) - 1] = '\0';

    tomatoLog[MAX_TOMATOES-1].gasValue = currentGas;
  }

  // Print the analysis results to serial monitor
  Serial.println("\n----- TOMATO ANALYSIS RESULTS -----");
  Serial.println(analysisResults);
  Serial.println("-----------------------------------");
```

```
    tomatoLog[MAX_TOMATOES-1].gasValue = currentGas;
  }

  // Print the analysis results to serial monitor
  Serial.println("\n----- TOMATO ANALYSIS RESULTS -----");
  Serial.println(analysisResults);
  Serial.println("-----------------------------------");

  // Parse results to determine if tomato is accepted and its ripeness
  tomatoAccepted = analysisResults.indexOf("REJECTED") == -1;
  tomatoIsRipe = analysisResults.indexOf("Ripe (Red-Orange)") >= 0;

  // Start the tomato processing sequence
  startTomatoProcessing();

  server.send(200, "text/plain", "Results received");
 } else {
  server.send(400, "text/plain", "No data received");
 }

}
```

CODES

ESP32-CAM.INO

TEST2ESPCAM.ino

```ino
#include "esp_camera.h"
#include <WiFi.h>
#include <WiFiClient.h>
#include <WebServer.h>
#include <ESPmDNS.h>
#include <HTTPClient.h>

#define CAMERA_MODEL_AI_THINKER // Make sure this matches your camera module

#if defined(CAMERA_MODEL_AI_THINKER)
#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27
#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22
#else
#error "Camera model not selected"
#endif

// Flash LED
#define FLASH_LED_PIN 4

const char* ssid = "BSIT-S2A-G3";
const char* password = "12345678";
const char* mainESP32Address = "http://192.168.72.244/results"; // IP of the main ESP32

WebServer server(80);

// Function to analyze tomato for dark spots
bool analyzeDarkSpots(camera_fb_t *fb) {
  if (!fb) return false;

  // Convert the JPEG to raw pixels for analysis
  // This is a simplified version - in reality, you'd need to decode the JPEG
  uint8_t *buf = fb->buf;
  size_t len = fb->len;

  // Count dark pixels - this is a simplified approach
  int darkPixelCount = 0;
  int threshold = 50; // Adjust based on lighting conditions
  int sampleSize = 100; // Number of pixels to sample (for speed)
```

TEST2ESPCAM.ino

```ino
  // Sample pixels throughout the image
  for (int i = 0; i < sampleSize; i++) {
    int idx = (i * len / sampleSize) % len;
    if (idx < len && buf[idx] < threshold) {
      darkPixelCount++;
    }
  }

  // If more than 20% of sampled pixels are dark, consider as having too many dark spots
  return (darkPixelCount > (sampleSize * 0.2));
}

// Function to analyze tomato ripeness
String analyzeRipeness(camera_fb_t *fb) {
  if (!fb) return "Unknown";

  // This is a simplified approach - in reality, you'd need color analysis
  uint8_t *buf = fb->buf;
  size_t len = fb->len;

  // Count red/orange vs yellow/green pixels
  int redOrangeCount = 0;
  int yellowGreenCount = 0;
  int sampleSize = 100;

  // Simplified color analysis (actual implementation would decode JPEG and analyze RGB values)
  for (int i = 0; i < sampleSize; i++) {
    int idx = (i * len / sampleSize) % len;
    if (idx + 2 < len) {
      // Simplified color detection - this would be more complex in real implementation
      uint8_t r = buf[idx];
      uint8_t g = buf[idx + 1];
      uint8_t b = buf[idx + 2];

      if (r > g + 50) { // More red than green - reddish
        redOrangeCount++;
      } else if (g > r - 20) { // More green - yellowish/greenish
        yellowGreenCount++;
      }
    }
  }

  if (redOrangeCount > yellowGreenCount) {
    return "Ripe (Red-Orange)";
  } else {
    return "Unripe (Yellow-Green)";
  }
}

void captureAndAnalyze() {
  // Turn on flash LED
  digitalWrite(FLASH_LED_PIN, HIGH);
  delay(100); // Short flash stabilization
```

CODES

ESP32-CAM.INO

TEST2ESPCAM.ino

```
108   // First image capture
109   camera_fb_t *fb1 = esp_camera_fb_get();
110
111   if (!fb1) {
112     Serial.println("First camera capture failed");
113     digitalWrite(FLASH_LED_PIN, LOW);
114     return;
115   }
116
117   // Analyze first image
118   bool hasDarkSpots1 = analyzeDarkSpots(fb1);
119   String ripeness1 = analyzeRipeness(fb1);
120
121   // Wait for main ESP32 to rotate the tomato
122   delay(15000); // 15 seconds for rotation
123
124   // Second image capture
125   camera_fb_t *fb2 = esp_camera_fb_get();
126   digitalWrite(FLASH_LED_PIN, LOW); // Turn off flash after capture
127
128   if (!fb2) {
129     Serial.println("Second camera capture failed");
130     esp_camera_fb_return(fb1);
131     return;
132   }
133
134   // Analyze second image
135   bool hasDarkSpots2 = analyzeDarkSpots(fb2);
136   String ripeness2 = analyzeRipeness(fb2);
137
138   // Combine results
139   bool rejected = hasDarkSpots1 && hasDarkSpots2; // Reject only if both images show dark spots
140   String ripenessResult = (ripeness1 == ripeness2) ? ripeness1 : "Mixed results: " + ripeness1 + " and " + ripeness2;
141
142   // Create result string
143   String result = "Tomato Analysis Results:\n";
144   result += "Dark Spots: " + String(rejected ? "Too many (REJECTED)" : "Few or none (ACCEPTED)") + "\n";
145   result += "Ripeness: " + ripenessResult;
146
147   // Log results
148   Serial.println(result);
149
150   // Send results to main ESP32
151   HTTPClient http;
152   http.begin(mainESP32Address);
153   http.addHeader("Content-Type", "text/plain");
154   int httpResponseCode = http.POST(result);
155
156   if (httpResponseCode > 0) {
157     Serial.print("HTTP Response code: ");
158     Serial.println(httpResponseCode);
159   }
160   else {
161     Serial.print("Error code: ");
```

TEST2ESPCAM.ino

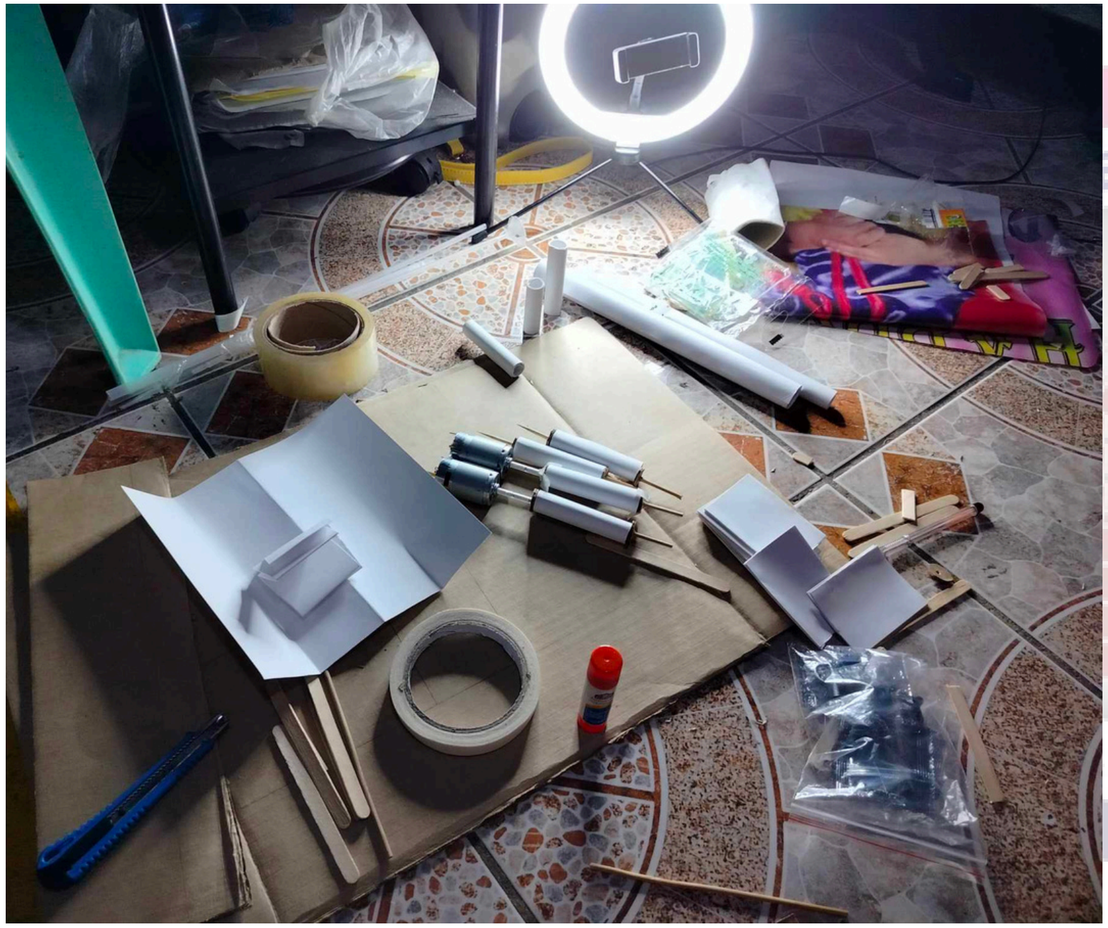
```
161     Serial.print("Error code: ");
162     Serial.println(httpResponseCode);
163   }
164   http.end();
165
166   // Free the memory
167   esp_camera_fb_return(fb1);
168   esp_camera_fb_return(fb2);
169 }
170
171 void handleCapture() {
172   server.send(200, "text/plain", "Capturing images and analyzing tomato...");
173   captureAndAnalyze();
174 }
175
176 void setup() {
177   Serial.begin(115200);
178
179   // Configure flash LED pin
180   pinMode(FLASH_LED_PIN, OUTPUT);
181   digitalWrite(FLASH_LED_PIN, LOW);
182
183   // Camera configuration
184   camera_config_t config;
185   config.ledc_channel = LEDC_CHANNEL_0;
186   config.ledc_timer = LEDC_TIMER_0;
187   config.pin_d0 = Y2_GPIO_NUM;
188   config.pin_d1 = Y3_GPIO_NUM;
189   config.pin_d2 = Y4_GPIO_NUM;
190   config.pin_d3 = Y5_GPIO_NUM;
191   config.pin_d4 = Y6_GPIO_NUM;
192   config.pin_d5 = Y7_GPIO_NUM;
193   config.pin_d6 = Y8_GPIO_NUM;
194   config.pin_d7 = Y9_GPIO_NUM;
195   config.pin_xclk = XCLK_GPIO_NUM;
196   config.pin_pclk = PCLK_GPIO_NUM;
197   config.pin_vsync = VSYNC_GPIO_NUM;
198   config.pin_href = HREF_GPIO_NUM;
199   config.pin_sscb_sda = SIOD_GPIO_NUM;
200   config.pin_sscb_scl = SIOC_GPIO_NUM;
201   config.pin_pwdn = PWDN_GPIO_NUM;
202   config.pin_reset = RESET_GPIO_NUM;
203   config.xclk_freq_hz = 20000000;
204   config.pixel_format = PIXFORMAT_JPEG;
205   config.frame_size = FRAMESIZE_VGA; // Use VGA for better analysis
206   config.jpeg_quality = 10; // Higher quality for better analysis
207   config.fb_count = 2; // Use 2 framebuffers for better performance
208
209   // Initialize camera
210   esp_err_t err = esp_camera_init(&config);
211   if (err != ESP_OK) {
212     Serial.printf("Camera init failed with error 0x%x", err);
213     return;
```

CODES

ESP32-CAM.INO

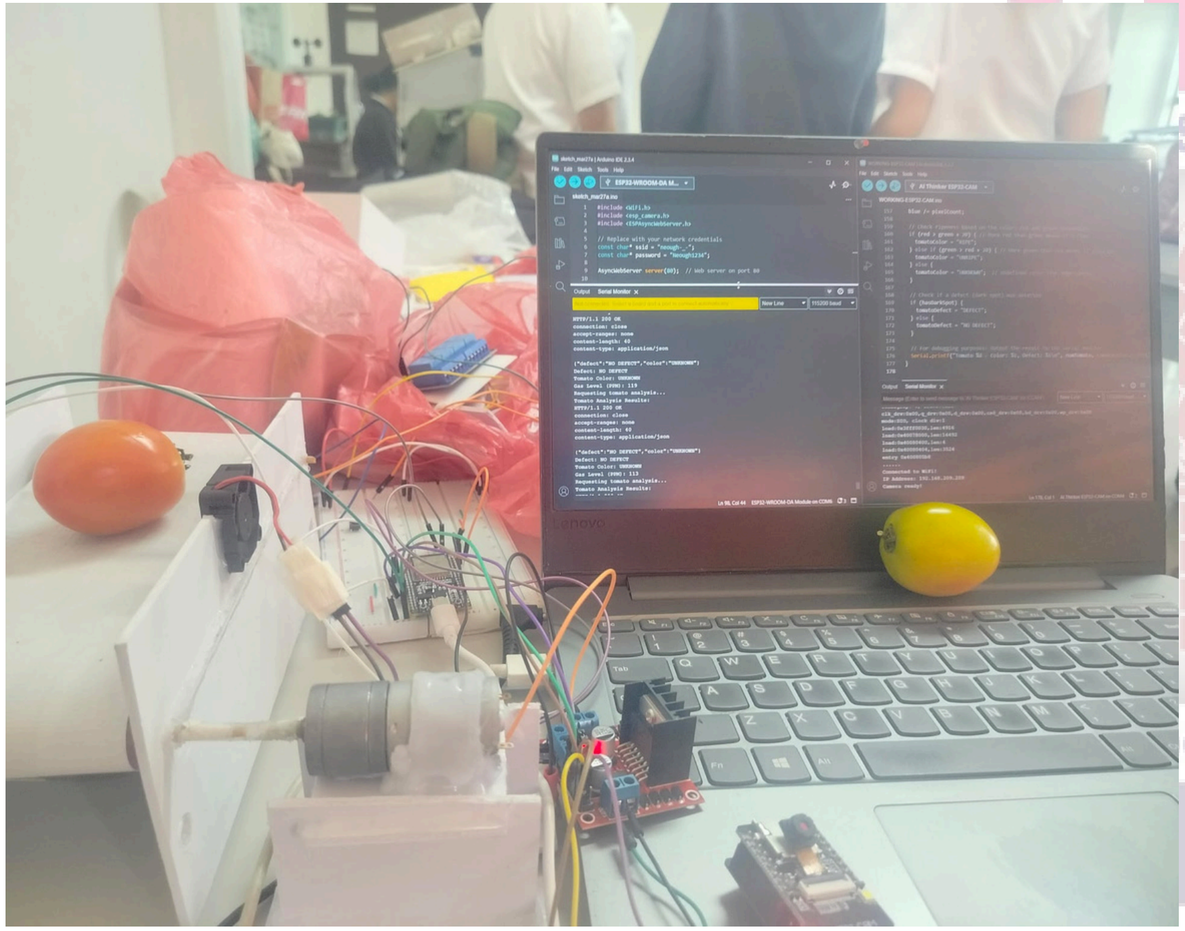
```
208
209   // Initialize camera
210   esp_err_t err = esp_camera_init(&config);
211   if (err != ESP_OK) {
212     Serial.printf("Camera init failed with error 0x%x", err);
213     return;
214   }
215
216   // Set higher contrast and saturation for better color detection
217   sensor_t * s = esp_camera_sensor_get();
218   s->set_contrast(s, 2);      // Increase contrast
219   s->set_saturation(s, 2);    // Increase saturation for better color detection
220   s->set_brightness(s, 1);    // Slightly increase brightness
221
222   // Connect to WiFi
223   WiFi.begin(ssid, password);
224   while (WiFi.status() != WL_CONNECTED) {
225     delay(500);
226     Serial.print(".");
227   }
228   Serial.println("");
229   Serial.println("WiFi connected");
230   Serial.print("IP address: ");
231   Serial.println(WiFi.localIP());
232
233   // Set up server endpoints
234   server.on("/capture", HTTP_GET, handleCapture);
235   server.on("/", HTTP_GET, []() {
236     server.send(200, "text/plain", "ESP32-CAM Tomato Analyzer. Use /capture to start analysis.");
237   });
238
239   server.begin();
240   Serial.println("HTTP server started");
241 }
242
243 void loop() {
244   server.handleClient();
245 }
```

DOCUMENTATION





DOCUMENTATION





REFERENCES

- Microcontrollers Lab. (n.d.). MG995 Servo Motor pinout interfacing with Arduino - Features & examples. Retrieved March 6, 2025, from https://microcontrollerslab.com/mg995-servo-motor-pinout-interfacing-with-arduino-features-examples
- YouTube. (2023, May 12). How to use MQ135 gas sensor with ESP32 using BLYNK IoT Cloud & LCD Display. Retrieved March 6, 2025, from https://www.youtube.com/watch?v=CsMx5dFRMl4
- YouTube. (2023, October 15). ESP32-CAM video streaming web server with pan and tilt. Retrieved April 3, 2025, from https://www.youtube.com/watch?v=hbMzerhC4ds
- Arduino Forum. (n.d.). TCRT5000 circuit for ESP32 S3. Retrieved March 6 2025, from https://forum.arduino.cc/t/tcrt5000-circuit-for-esp32-s3/1318544